

DAQ_Winnipeg_City

December 8, 2024

1 Data Acquisition

- Winnipeg Housing Permit Inspection
- <https://data.Winnipeg.ca>

```
[ ]: import pandas as pd
      from sodapy import Socrata

      # Unauthenticated client only works with public data sets.
      # Note no username or password:
      client = Socrata("data.winnipeg.ca", "xxx")

      # Getting data
      wpg_inspect_defects = client.get("ytpf-7jye")

      # Convert to pandas DataFrame
      wpg_inspect_defects_df = pd.DataFrame.from_records(wpg_inspect_defects)

[ ]: wpg_inspect_defects_df.head()

[ ]: wpg_inspect_defects_df.tail()

[ ]: wpg_inspect_defects_df.columns

[ ]: wpg_inspect_defects_df.info()

[ ]: wpg_inspect_defects_df['inspection_count'] = pd.
      ↪to_numeric(wpg_inspect_defects_df['inspection_count'],errors = 'coerce')
      wpg_inspect_defects_df['permit_count'] = pd.
      ↪to_numeric(wpg_inspect_defects_df['permit_count'],errors = 'coerce')
      wpg_inspect_defects_df['defect_count'] = pd.
      ↪to_numeric(wpg_inspect_defects_df['defect_count'],errors = 'coerce')
      wpg_inspect_defects_df['year'] = pd.
      ↪to_numeric(wpg_inspect_defects_df['year'],errors = 'coerce')

[ ]: wpg_inspect_defects_df.info()
```

```
[ ]: wpg_inspect_defects_df['inspection_type'].unique()

[ ]: wpg_inspect_defects_df['year'].unique()

[ ]: wpg_inspect_defects_df['ward'].unique()

[ ]: #wpg_inspect_defects_df_drop = wpg_inspect_defects_df.drop('location', axis=1,
    ↪inplace=False)

wpg_inspect_defects_df.drop('location', axis=1, inplace=True)
wpg_inspect_defects_df.head()

[ ]: wpg_inspect_defects_df.shape

[ ]: wpg_inspect_defects_df_pv = pd.pivot_table(wpg_inspect_defects_df,
    ↪values=['inspection_count', 'defect_count'],
    index=['ward', 'year'], columns=['inspection_type'])
wpg_inspect_defects_df_pv

[ ]: wpg_inspect_defects_df_pv.shape

[ ]: wpg_inspect_defects_df_pv.columns.map('_'.join)

[ ]: wpg_inspect_defects_df_pv.columns = wpg_inspect_defects_df_pv.columns.map('_'.
    ↪join)
wpg_inspect_defects_df_pv

[ ]: wpg_inspect_defects_df_pv = wpg_inspect_defects_df_pv.reset_index()
wpg_inspect_defects_df_pv

[ ]: wpg_inspect_defects_df_pv['total_inspection_count'] =
    ↪wpg_inspect_defects_df_pv['inspection_count_Residential Building'] +
    ↪wpg_inspect_defects_df_pv['inspection_count_Residential Electrical'] +
    ↪wpg_inspect_defects_df_pv['inspection_count_Residential Plumbing']
wpg_inspect_defects_df_pv['total_defect_count'] =
    ↪wpg_inspect_defects_df_pv['defect_count_Residential Building'] +
    ↪wpg_inspect_defects_df_pv['defect_count_Residential Electrical'] +
    ↪wpg_inspect_defects_df_pv['defect_count_Residential Plumbing']
wpg_inspect_defects_df_pv['total_defect_ratio'] =
    ↪wpg_inspect_defects_df_pv['total_defect_count']/
    ↪wpg_inspect_defects_df_pv['total_inspection_count']
wpg_inspect_defects_df_pv

[ ]: wpg_inspect_defects_target =
    ↪wpg_inspect_defects_df_pv[['ward', 'year', 'total_defect_ratio']]
wpg_inspect_defects_target
```

```
[ ]: wpg_inspect_defects_target.info()
```

```
[ ]: import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(16, 8))
sns.set_theme(style="whitegrid")

# Plot the responses for different events and regions
sns.lineplot(x="year", y="total_defect_ratio",
             hue="ward",
             data=wp_g_inspect_defects_target,
             palette=sns.color_palette("deep", 20),
             linewidth=2.5)
plt.show()
```

```
[ ]: #Census 2006 - Dwelling
wp_g_census_2006_dwelling = client.get("cem6-ktqa")

# Convert to pandas DataFrame
wp_g_census_2006_dwelling_df = pd.DataFrame.
↳from_records(wp_g_census_2006_dwelling)
```

```
[ ]: wp_g_census_2006_dwelling_df.head()
```

```
[ ]: wp_g_census_2006_dwelling_df.tail()
```

```
[ ]: wp_g_census_2006_dwelling_df.columns
```

```
[ ]: wp_g_census_2006_dwelling_df.info()
```

```
[ ]: wp_g_census_2006_dwelling_df['dwelling_cost_owner_average_value_dwelling'] = pd.
↳to_numeric(wp_g_census_2006_dwelling_df['dwelling_cost_owner_average_value_dwelling'],errors
↳='coerce')
wp_g_census_2006_dwelling_df['dwelling_occupied_average_rooms'] = pd.
↳to_numeric(wp_g_census_2006_dwelling_df['dwelling_occupied_average_rooms'],errors
↳='coerce')
wp_g_census_2006_dwelling_df['dwelling_condition_regular_maintenance'] = pd.
↳to_numeric(wp_g_census_2006_dwelling_df['dwelling_condition_regular_maintenance'],errors
↳='coerce')
wp_g_census_2006_dwelling_df['dwelling_condition_major_repairs'] = pd.
↳to_numeric(wp_g_census_2006_dwelling_df['dwelling_condition_major_repairs'],errors
↳='coerce')
```

```
[ ]: wp_g_census_2006_dwelling_df.info()
```

```
[ ]: wpg_census_2006_dwelling_df_sel =
↳wpg_census_2006_dwelling_df[['boundary_type','boundary_name','dwelling_cost_owner_average_v
wpg_census_2006_dwelling_df_sel

[ ]: wpg_census_2006_dwelling_df_sel = wpg_census_2006_dwelling_df_sel.
↳loc[wpg_census_2006_dwelling_df_sel['boundary_type'] == 'Ward']

[ ]: wpg_census_2006_dwelling_df_sel['boundary_name'].unique()

[ ]: ax = sns.barplot(y="boundary_name",
↳x="dwelling_cost_owner_average_value_dwelling", facecolor="blue",
↳data=wpg_census_2006_dwelling_df_sel)

[ ]: ax = sns.barplot(y="boundary_name", x="dwelling_occupied_average_rooms",
↳facecolor="green", data=wpg_census_2006_dwelling_df_sel)

[ ]: ax = sns.barplot(y="boundary_name", x="dwelling_condition_regular_maintenance",
↳facecolor="grey", data=wpg_census_2006_dwelling_df_sel)

[ ]: ax = sns.barplot(y="boundary_name", x="dwelling_condition_major_repairs",
↳facecolor="orange", data=wpg_census_2006_dwelling_df_sel)

[ ]: # Census - Households
wpg_census_households = client.get("nmk5-uwfw")

# Convert to pandas DataFrame
wpg_census_households_df = pd.DataFrame.from_records(wpg_census_households)

[ ]: wpg_census_households_df.head()

[ ]: wpg_census_households_df.columns

[ ]: wpg_census_households_df_sel = wpg_census_households_df.
↳loc[wpg_census_households_df['boundary_type'] == 'Ward']
wpg_census_households_df_sel

[ ]: wpg_census_households_df_sel =
↳wpg_census_households_df_sel[['census_year','boundary_name','income_average','size_average_
wpg_census_households_df_sel

[ ]: wpg_census_households_df_sel['income_average'] = pd.
↳to_numeric(wpg_census_households_df_sel['income_average'],errors = 'coerce')
wpg_census_households_df_sel['size_average_person'] = pd.
↳to_numeric(wpg_census_households_df_sel['size_average_person'],errors =
↳'coerce')
ax = sns.barplot(y="boundary_name", x="size_average_person", hue="census_year",
↳data=wpg_census_households_df_sel)
```

```
[ ]: ax = sns.barplot(y="boundary_name", x="income_average", hue="census_year",  
↳data=wp_g_census_households_df_sel)
```

```
[ ]: #Census - Labour Force Activities  
wp_g_census_labour_act = client.get("pcyz-4rbx")  
  
# Convert to pandas DataFrame  
wp_g_census_labour_act_df = pd.DataFrame.from_records(wp_g_census_labour_act)  
wp_g_census_labour_act_df.head()
```

```
[ ]: wp_g_census_labour_act_df["boundary_type"].unique()
```

```
[ ]: print(wp_g_census_labour_act_df.columns)
```

```
[ ]: wp_g_census_labour_act_df.head()
```

```
[ ]:
```